

TaxHub — Architecture

TaxHub — Architecture

1. Current features

Web routes (web/app.py, port 5011)

Jurisdiction coverage

2. Component architecture

3. Scrape & versioning flow

4. Graph data model (Neo4j)

5. Agents + Graph-RAG — the assistant

6. Environment variables

7. Deployment

7.1 Local dev (user-owned Neo4j, no sudo)

7.2 Coolify / Docker Compose

7.3 AuraDB (recommended production graph)

8. Testing

9. Roadmap — what to do next

Tax-form finding and tax-law traceability for a fund-management back office. TaxHub's primary job is to help the back office find the **correct tax form** to file across the jurisdictions JTC Group operates in; secondarily it scrapes official tax documents, captures **every version**, detects and explains **every change**, traces guidance back to the **primary law** it interprets, and answers questions over the resulting citation/change graph.

This document describes the implemented system: components, data model, data flow, the LangGraph agent orchestrator + graph-RAG retrieval, configuration, and deployment (local → Coolify → AuraDB). For a quick start see the top-level [README.md](#).

1. Current features

Area	Feature	Where
Forms	Config-driven form catalogue, 26 jurisdictions	<code>config/tax_forms.yaml</code> , <code>ingest/forms.py</code>
	Form metadata + PDF download / discover-all over authority indexes	<code>ingest/forms.py</code> , <code>ingest/scrapers/</code>
	Forms Tree taxonomy (jurisdiction → category → type → form)	<code>ingest/forms.py:forms_tree</code>

Area	Feature	Where
Ingest	Config-driven document catalogue	<code>config/tax_sources.yaml</code> , <code>ingest/cli.py</code>
	HTTP + headless-browser fetch (JS/UA-gated sites)	<code>ingest/fetch.py</code> , <code>ingest/scrapers/</code>
	HTML + PDF text extraction, normalised + SHA-256 hashed	<code>ingest/fetch.py</code>
Versioning	Immutable version appended only when the content hash changes	<code>storage/*</code> , <code>ingest/cli.py</code>
	Consecutive-version diffs (added/removed chars + diff text)	<code>ingest/cli.py</code> , <code>rag/llm.py</code>
AI	Grok plain-English change summary + back-office impact	<code>rag/llm.py</code>
	Citation extraction (guidance → statutory instruments)	<code>rag/llm.py</code>
	Graceful degradation when no <code>XAI_API_KEY</code> is set	<code>rag/llm.py</code> , <code>rag/retrieval.py</code> , <code>agents/orchestrator.py</code>
Agents	LangGraph tool-calling orchestrator routing to 4 specialist agents	<code>agents/orchestrator.py</code> , <code>agents/tools.py</code>
	SSE token + tool-step streaming to the UI	<code>agents/sse.py</code> , <code>agents/orchestrator.py</code>
Storage	Backend-neutral <code>Storage</code> interface	<code>storage/base.py</code>
	Neo4j graph backend (default)	<code>storage/neo4j_store.py</code>
	SQLite / Postgres relational backend	<code>storage/sqlite_store.py</code>
	One-line backend switch via <code>DATA_STORAGE</code>	<code>storage/__init__.py</code>
	SQLite → Neo4j backfill migration	<code>scripts/migrate_sqlite_to_neo4j.py</code>
Retrieval	Full-text seed → graph expansion → Grok answer with cited sources	<code>rag/retrieval.py</code>
	Vector + hybrid (RRF) retrievers over fastembed chunk embeddings	<code>rag/retrieval.py</code> , <code>rag/embeddings.py</code>
	Chunk-embedding backfill	<code>scripts/embed_backfill.py</code>
Web app	FastHTML 3-pane agentic app: Forms Tree, AI chat, changes/PDF viewer	<code>web/app.py</code>
CLI		<code>ingest/cli.py</code>

Area	Feature	Where
	<pre>--list / --jurisdiction / --all / --changes / --stats / --no-ai</pre>	
Ops	Healthcheck, Dockerfile, Coolify-labelled compose, user-owned local Neo4j	<code>docker-compose.yaml</code> , <code>scripts/neo4j_local.sh</code>
Tests	Cross-backend storage contract + no-network RAG smoke	<code>tests/</code>
Evals	Retriever comparison harness over a ground-truth set	<code>evals/</code>

Web routes (`web/app.py` , port 5011)

Route	Purpose
<code>/health</code>	Liveness probe (used by the compose healthcheck)
<code>/login</code> , <code>/logout</code>	Session auth (admin seeded by <code>init_db</code>)
<code>/</code>	3-pane app: Forms Tree + AI Assistant chat + changes newsfeed
<code>/chat</code> (POST)	SSE stream from the LangGraph orchestrator (or shortcut routing)
<code>/dashboard</code>	Corpus stats + jurisdictions with document counts
<code>/jurisdictions</code> , <code>/jurisdiction/{code}</code>	Jurisdiction list / documents in a jurisdiction
<code>/documents</code> , <code>/upload</code>	Stored-PDF browser with filters; upload a PDF to the volume
<code>/document/{doc_id}</code>	Version history, changes, citations for a document
<code>/form/{form_id}</code> , <code>/form-pdf/{form_id}</code>	Form metadata + embedded PDF viewer
<code>/changes</code> , <code>/api/feed</code>	Recent changes across all jurisdictions / feed partial
<code>/help</code> , <code>/user-guide-pdf</code>	In-app help + the generated User Guide PDF

Jurisdiction coverage

The form catalogue (`config/tax_forms.yaml`) covers **26 jurisdictions** — the six live-MVP fund domiciles plus an expansion to all JTC Group office jurisdictions:

Live MVP: Jersey (JE), Guernsey (GG), Luxembourg (LU), Ireland (IE), Cayman Islands (KY), British Virgin Islands (VG).

Expansion: Isle of Man (IM), Mauritius (MU), Malta (MT), Cyprus (CY), Netherlands (NL), Switzerland (CH), United Kingdom (GB), Germany (DE), Austria (AT), Poland (PL), United States (US), Hong Kong (HK), Singapore (SG), Malaysia (MY), New Zealand (NZ), United Arab Emirates (AE), Bahamas (BS), Brazil (BR), South Africa (ZA), Bermuda (BM).

Each form is labelled by **filing type** — **downloadable** (a PDF to complete), **online** (web portal / e-file), or **reference** (guidance, not a fileable form). Most jurisdictions file online, but a subset publishes **downloadable PDFs** — notably the US (IRS), Hong Kong, New Zealand, Poland, Austria, Bermuda, Luxembourg, and Guernsey — which TaxHub fetches and stores locally for the in-app PDF viewer.

The document/legislation catalogue (`config/tax_sources.yaml`) used for versioning + graph-RAG currently tracks the original six MVP jurisdictions.

2. Component architecture

```
flowchart TB
    subgraph Sources["Official sources"]
        SRC["Tax authorities across 26<br/>JTC Group jurisdictions<br/>(forms indexes, legislation,<br/>guidance portals)"]
    end

    subgraph Ingest["Ingestion (ingest/)"]
        FCFG["config/tax_forms.yaml<br/>(26-jurisdiction form catalogue)"]
        SCFG["config/tax_sources.yaml<br/>(document catalogue)"]
        FORMS["ingest/forms.py<br/>form scrape + Forms Tree"]
        FETCH["ingest/fetch.py + scrapers/<br/>fetch + extract + hash"]
        ORCH["ingest/cli.py<br/>scraper CLI"]
        AI["rag/llm.py<br/>Grok summary + citations"]
    end

    subgraph StorageLayer["Storage (backend-neutral)"]
        BASE["storage/base.py<br/>Storage ABC"]
        SHIM["taxstore.py<br/>compat shim → get_store()"]
        NEO4["storage/neo4j_store.py"]
        SQL["storage/sqlite_store.py"]
    end

    subgraph Backends["Databases"]
        NEO4J["Neo4j / AuraDB"]
        SQLITE["SQLite / Postgres"]
    end

    subgraph Serve["Serving (web/ + agents/ + rag/)"]
        APP["web/app.py<br/>FastHTML 3-pane app"]
        AGENTS["agents/orchestrator.py<br/>LangGraph agent + 4 tools"]
        RAG["rag/retrieval.py<br/>fulltext/vector/hybrid + answer()"]
    end
```

```

    EMB["rag/embeddings.py<br/>fastembed chunks"]
end

USER(["Back-office user"])
GROK[["xAI Grok API"]]

SRC --> FETCH
FCFG --> FORMS
SCFG --> ORCH
ORCH --> FETCH
FORMS --> FETCH
FETCH --> ORCH
ORCH --> AI
AI -.-> GROK
ORCH --> SHIM
FORMS --> SHIM
EMB --> SHIM
SHIM --> BASE
BASE --> NEO
BASE --> SQL
NEO --> NEO4J
SQL --> SQLITE
USER --> APP
APP --> AGENTS
AGENTS --> RAG
AGENTS -.-> GROK
RAG --> EMB
RAG --> SHIM
RAG -.-> GROK
APP --> SHIM

```

DATA_STORAGE selects exactly one backend at runtime; both implement the same **Storage** interface, so no caller (app, agents, retrieval, CLI) knows which database is live. The top-level **taxstore.py** is a thin compat shim that re-exports the active store from **storage.get_store()**.

3. Scrape & versioning flow

How one document moves from source to a stored, diffed, AI-summarised version:

```

sequenceDiagram
    participant CLI as ingest/cli.py
    participant F as ingest/fetch.py
    participant S as Storage backend
    participant AI as rag/llm.py (Grok)

    CLI->>S: start_run(jurisdiction)
    CLI->>F: fetch(url, mode=http|browser)
    F->>F: extract text (html|pdf) + normalise
    F->>F: sha256(normalised text)

```

```

F-->>CLI: text, content_hash
CLI->>S: latest_version(doc_id)
alt hash unchanged
    CLI->>S: mark_checked(doc_id)
else hash changed (or first capture)
    CLI->>S: add_version(doc_id, hash, text)
    CLI->>CLI: diff(prev_text, new_text)
    CLI->>AI: summarise change + impact
    AI->>CLI: ai_summary, ai_impact
    CLI->>S: record_change(from_v, to_v, diff, ai_*)
    CLI->>AI: extract_citations(text)
    AI->>CLI: instruments + locators
    CLI->>S: add_citations(doc_id, version, citations)
end
CLI->>S: finish_run(changed, new, changed, errors)

```

Versions are **immutable and append-only** — the audit trail of how the law changed. A change row is written only on an actual content-hash difference.

4. Graph data model (Neo4j)

```

graph LR
    J["(:Jurisdiction)<br/>code, name, region, authority"]
    D["(:Document)<br/>uid, doc_key, title, reference, url"]
    V["(:Version)<br/>uid, version_no, content_hash, text"]
    C["(:Change)<br/>uid, change_type, diff, ai_summary, ai_impact"]
    I["(:Instrument)<br/>key, name"]
    CH["(:Chunk)<br/>ord, chunk_text, embedding"]
    F["(:Form)<br/>uid, form_key, title, filing_type, file_path"]

    J -- HAS_DOCUMENT --> D
    J -- HAS_FORM --> F
    D -- HAS_VERSION --> V
    D -- CURRENT_VERSION --> V
    D -- HAS_CHANGE --> C
    C -- FROM_VERSION --> V
    C -- TO_VERSION --> V
    V -- "CITES {locator, snippet}" --> I
    V -- HAS_CHUNK --> CH

```

Design notes:

- **Stable integer ids** (`uid`) are minted from a (`:Counter`) node, so `/document/{id}` and `/form/{id}` URLs work identically across backends and the deprecated `id()` function is avoided — portable straight to AuraDB.
- **Citations are first-class edges** (`(:Version)-[:CITES]->(:Instrument)`), which is exactly what graph-RAG traverses.

- **Forms are first-class nodes** (`(:Jurisdiction)-[:HAS_FORM]->(:Form)`), classified by category + form_type to build the Forms Tree, and distinct from legislation/guidance `Document` nodes.
- **Chunk vectors** (`(:Version)-[:HAS_CHUNK]->(:Chunk)`) carry per-passage embeddings for vector/hybrid retrieval, backed by a Neo4j native `VECTOR INDEX` (cosine).
- **Null properties are re-materialised** on read (Neo4j omits null keys; the app expects every column present) so the two backends are behaviourally identical.
- **Version-tolerant DDL**: `init_db` tries 5.x syntax (`REQUIRE` , `CREATE FULLTEXT INDEX`) and falls back to 4.x (`ASSERT` , `db.index.fulltext` procedure), so the same code runs on local Neo4j 4.x and AuraDB 5.x.

The relational backend stores the same shape as tables: `jurisdictions` , `tax_documents` , `document_versions` , `document_changes` , `citations` , `version_chunks` , `tax_forms` , `scrape_runs` , `users` , plus chat history (`chat_sessions` , `chat_messages`).

5. Agents + Graph-RAG — the assistant

The web app's chat routes every message to a **LangGraph tool-calling orchestrator** (`agents/orchestrator.py` , `create_react_agent`). The orchestrator picks the right specialist agent — exposed as the four tools in `agents/tools.py` — and streams tokens and tool steps back over SSE:

Tool	Job
<code>document_agent</code>	Find the correct tax FORM(s) for a need (the primary use case)
<code>law_agent</code>	Tax-LAW Q&A via graph-RAG over the tracked corpus, with citations
<code>metadata_agent</code>	Structured lookup: forms / deadlines for a jurisdiction or category
<code>changes_agent</code>	Recent changes to tracked documents

`law_agent` delegates to graph-RAG retrieval:

```

flowchart LR
    Q(["Question"]) --> SEED
    subgraph Retrieve["rag.retrieval – fulltext / vector / hybrid"]
        SEED["1. Seed<br/>full-text (Neo4j Lucene / SQLite scan)<br/>and/or vector (fastembed chunks)<br/>fused by reciprocal-rank (hybrid)"]
        EXP["2. Graph expansion<br/>cited instruments + latest change"]
        SEED --> EXP
    end
    end
    EXP --> CTX["Numbered, grounded context blocks"]
    CTX --> GEN["rag.retrieval.answer()<br/>Grok generation"]
    GEN --> A(["Answer + cited sources"])
    GEN -. "no XAI_API_KEY" .-> DEG["Degrade: return sources only"]

```

Retrieval and generation are **deliberately separated** (`Retriever` ABC + `answer()`). Three retrievers implement the ABC — `GraphFullTextRetriever` , `VectorRetriever` , and `HybridRetriever` — selected by `RAG_RETRIEVER` (default `hybrid`). **Embeddings are local fastembed** (`BAAI/bge-small-en-v1.5` , 384-dim; `rag/embeddings.py`), chunked per passage and stored as `(:Chunk)` nodes / `version_chunks` rows. Vector and hybrid retrievers **degrade to full-text** when no embeddings/chunks are present, so the assistant works with or without a backfill (`scripts/embed_backfill.py`).

6. Environment variables

Copy `.env.example` → `.env` (gitignored). All variables:

Variable	Required	Default	Purpose
<code>DATA_STORAGE</code>	yes	<code>neo4j</code>	Active backend: <code>neo4j</code> or <code>sqlite</code>
<code>DB_URL</code>	sqlite mode	<code>sqlite:///taxhub.db</code>	SQLite/Postgres DSN; also the migration source
<code>NEO4J_URI</code>	neo4j mode	<code>bolt://localhost:7687</code>	Bolt URI. AuraDB: <code>neo4j+s://<id>.databases.neo4j.io</code>
<code>NEO4J_USER</code>	neo4j mode	<code>neo4j</code>	Neo4j username
<code>NEO4J_PASSWORD</code>	neo4j mode	—	Neo4j password
<code>NEO4J_DATABASE</code>	no	<code>neo4j</code>	Target database name
<code>XAI_API_KEY</code>	no	—	xAI Grok key; absent → AI features degrade gracefully
<code>XAI_BASE_URL</code>	no	<code>https://api.x.ai/v1</code>	OpenAI-compatible base URL
<code>GROK_MODEL</code>	no	<code>grok-4-1-fast-reasoning</code>	Model id
<code>LLM_PROVIDER</code>	no	<code>xai</code>	Provider selector
<code>RAG_RETRIEVER</code>	no	<code>hybrid</code>	Retriever: <code>hybrid</code> , <code>vector</code> , or <code>fulltext</code>
<code>EMBEDDINGS</code>	no	<code>fastembed</code>	Embedding provider: <code>fastembed</code> or <code>openai</code>
<code>EMBED_MODEL</code>	no	<code>BAAI/bge-small-en-v1.5</code>	Embedding model id
<code>EMBED_DIM</code>	no	<code>384</code>	Embedding dimensions (must match the model)

Variable	Required	Default	Purpose
<code>ADMIN_EMAIL</code>	yes	<code>admin@example.com</code>	Seeded admin login
<code>ADMIN_PASSWORD</code>	yes	<i>(set in env)</i>	Seeded admin password (set a strong value in prod)
<code>APP_SECRET</code>	yes (prod)	<i>(set in env)</i>	Session signing secret (set a random value in prod)
<code>TAXHUB_PUBLIC</code>	no	<code>0</code>	<code>1</code> disables login (public/demo mode)
<code>PORT</code>	no	<code>5011</code>	Web app port

7. Deployment

7.1 Local dev (user-owned Neo4j, no sudo)

```
scripts/neo4j_local.sh setup      # one-time: build config + set initial password
scripts/neo4j_local.sh start      # bolt://localhost:7687, http://localhost:7474
python3.12 scripts/migrate_sqlite_to_neo4j.py --wipe  # backfill the graph from SQLite
python3.12 -m uvicorn web.app:app --port 5011
```

Run on SQLite instead with `DATA_STORAGE=sqlite` (no Neo4j needed).

7.2 Coolify / Docker Compose

The `web` service is Coolify-labelled (`coolify.managed` , `coolify.port=5011`) with a `/health` healthcheck. The `scrape` service (profile `tools`) shares the data volume for on-demand/scheduled runs.

```
docker compose up -d web
docker compose run --rm scrape --all      # populate / refresh the corpus
```

Point `NEO4J_URI` at AuraDB (recommended) **or** bring up the bundled Neo4j:

```
docker compose --profile neo4j up -d      # self-hosted neo4j:5.26-community
```

7.3 AuraDB (recommended production graph)

```
flowchart LR
    subgraph Coolify["Coolify host"]
        WEB["web (FastHTML :5011)"]
        SCRAPE["scrape (cron/on-demand)"]
    end
    end
```

```

AURA[("Neo4j AuraDB<br/>neo4j+s://...")]
XAI[["xAI Grok API"]]

WEB -- "Bolt+TLS" --> AURA
SCRAPE -- "Bolt+TLS" --> AURA
WEB -. HTTPS .-> XAI
SCRAPE -. HTTPS .-> XAI

```

Steps:

1. Create a free/Pro AuraDB instance at <https://console.neo4j.io>; download the generated credentials.
2. Set in the Coolify environment: `DATA_STORAGE=neo4j NE04J_URI=neo4j+s://<id>.databases.neo4j.io NE04J_USER=<id> # see note below NE04J_PASSWORD=<generated> NE04J_DATABASE=<id> # see note below ( neo4j+s:// enables Bolt over TLS — required by Aura.)` **Gotcha:** on AuraDB *Free*, the username and the database name are **the instance id** (e.g. `05a16101`), **not** `neo4j` — connecting to database `neo4j` fails with `DatabaseNotFound`. Use the exact values from the credentials file Aura downloads at creation time (it's shown only once). On Professional the username/database are typically `neo4j`; either way, trust the downloaded credentials file.
3. Initialise the schema (idempotent, version-tolerant 4.x/5.x DDL):
`bash docker compose run --rm scrape --stats # or: python3.12 taxstore.py`
4. Backfill from an existing SQLite corpus, if any: `bash DB_URL=sqlite:///taxhub.db python3.12 scripts/migrate_sqlite_to_neo4j.py --wipe`
5. Schedule the scraper (e.g. Coolify cron) to build change history over time.

No application code changes between local Neo4j 4.x and AuraDB 5.x — only the four `NE04J_*` variables.

8. Testing

```
python3.12 -m pytest tests/ -q
```

- `tests/test_storage.py` — one contract suite parametrised over **both** backends, so `DATA_STORAGE` can't silently change behaviour. The Neo4j case skips if no Neo4j is reachable and cleans up its own `T9` subgraph.
- `tests/test_rag.py` — no-network RAG smoke (seeds a temp SQLite store, stubs Grok off, asserts retrieval + graceful degradation).

9. Roadmap — what to do next

Ordered by leverage:

1. **Deploy to AuraDB + Coolify** (§7.3) and point `taxhub.predictivelabs.ai` at it. First real production target.
2. **Schedule the scraper** (daily/weekly cron) so change history accrues — the product's core value is the *change* trail, which only exists once scrapes run repeatedly.
3. **Backfill embeddings at scale** — `VectorRetriever` / `HybridRetriever` and `scripts/embed_backfill.py` already exist (AuraDB native vector index, local fastembed). Run the backfill across the full corpus so `/chat` law answers are semantic-by-default rather than degrading to full-text.
4. **Change digest / alerts** — email or in-app digest of recent changes per jurisdiction (the "back-office alert" use case).
5. **Harden gated sources** — JS/UA-gated authority sites (e.g. Jersey, Guernsey) still under-fetch some forms; revisit the browser fetch strategy.
6. **Broaden the document/legislation catalogue** — `config/tax_sources.yaml` tracks only the six MVP jurisdictions for versioning + graph-RAG; extend it toward the 26 jurisdictions already in the form catalogue.
7. **Auth & multi-user** — move beyond the single seeded admin if more than the back-office team needs access.